

SHELL SCRIPTING

Definition

- A **shell** is a program that provides a command-line interface (CLI) to the operating system, you type commands, the shell interprets them and passes them to the kernel.
- A **shell script** is a plain text file which contains a sequence of shell commands, comments, variables and possibly programming-style constructs (loops, conditionals) that the shell executes.

Why Use Shell Scripting?

Shell scripting is useful for:

- **Automating repetitive tasks:** Instead of typing the same sequence of commands, you write them once in a script and run it.
- **System administration tasks:** Backups, file system cleanup, monitoring, configuring systems — all of these can be scripted.
- **Batch processing:** Running a series of commands in order, optionally checking results and branching based on conditions..
- **Consistency and reproducibility:** A script does the same thing each time (if written accordingly), reducing human error.

`/bin/bash`

This tells the system to run the script with

`#!/bin/bash`

`/bin/bash`

- This is the **path to the Bash shell** on most Linux systems.
- Bash (Bourne Again SHell) is one of the most common command interpreters in Unix/Linux.

Foundational Elements:

- **Shebang:** Begin scripts with `#!/bin/bash` to specify the interpreter.
- **Basic Commands:** Understand core Linux commands like `pwd`, `cd`, `ls`, `cp`, `mv`, `rm`.
- **Variables:** Declare and use variables to store data, such as `MY_VARIABLE="Hello"`.
- **Input/Output:** Handle user input and redirect output using commands and operators.
- **Control Flow:** Implement `if/else`, `case` statements, and `for/while` loops for conditional execution and iteration.
- **Functions:** Define and call functions to modularize code and promote reusability.
- **Positional Arguments:** Access command-line arguments passed to a script using `$1`, `$2`, etc.

Input/output Basics

`VARIABLE_NAME=value`

Ex:

```
#!/bin/bash
```

```
name="AUSHTMI"
```

```
echo "Hello, $name!"
```

Using Command Substitution

You can store the output of a command in a variable:

```
current_dir=$(pwd)
```

```
echo "You are in: $current_dir"
```

Read User Input into a Variable

```
read -p "Enter your age: " age
```

```
echo "You are $age years old."
```

```
readonly country="Bangladesh"(Constrant)
Export name="Aushtmi"(Global variable")
```

Print lines one after another
echo -e "Line1\nLine2"

Printf function
name="AUSHTMI"
age=25
printf "Name: %s\nAge: %d\n" "\$name" "\$age"
• You can read multiple variables at once

Multiline Comments:

```
<<'NOTE'
```

This script demonstrates:

1. Multi-line comments
2. Variables
3. User input and output

NOTE

Declaring Arrays

a) Using parentheses

```
fruits=("Apple" "Banana" "Orange")
```

```
colors=()
```

```
colors[0]="Red"
```

```
colors[1]="Green"
```

```
colors[2]="Blue"
```

Accessing Array

```
echo ${fruits[0]} # Apple  
echo ${fruits[1]} # Banana  
echo ${fruits[@]} #Access all the element  
echo ${#fruits[@]} # Find number of elements
```

Add a New Element

```
fruits=("Apple" "Banana")  
fruits+=("Orange")  
echo ${fruits[@]}
```

Remove an Element

```
fruits=("Apple" "Banana" "Mango")  
unset fruits[1]  
echo ${fruits[@]}
```

Insert an Element at Any Position (Recommended)

```
fruits=("Apple" "Banana" "Mango" "Orange")  
fruits=("${fruits[@]:0:2}" "Grapes" "${fruits[@]:2}")
```

if-else Statement

Basic Syntax

```
if [ condition ]  
then  
    # Statements if condition is true  
else  
    # Statements if condition is false  
fi
```

Comparison Operators

Numeric Comparison

Operator	Meaning
<code>-eq</code>	Equal to
<code>-ne</code>	Not equal to
<code>-gt</code>	Greater than
<code>-lt</code>	Less than
<code>-ge</code>	Greater than or equal to
<code>-le</code>	Less than or equal to

Case:

```
#!/bin/bash

echo "Enter a number between 1 to 3:"

read num

case $num in
    1)
        echo "You chose One" ;;
    2)
        echo "You chose Two" ;;
    3)
```

```
echo "You chose Three" ;;
```

```
echo "Invalid choice" ;;
```

```
esac
```

For loop:

```
for variable in list
```

```
do
```

```
    commands
```

```
done
```

Ex:

```
for i in {1..5}
```

```
do
```

```
    echo "Number: $i"
```

```
done
```

Another example:

```
for ((i=1; i<=5; i++))
```

```
do
```

```
    echo "Count: $i"
```

```
done
```

2. WHILE Loop

count=1

```
while [ $count -le 5 ]
```

```
do
```

```
    echo "Number: $count"
```

```
    ((count++))
```

```
done
```

Example 2: Read input until user types "stop"

```
#!/bin/bash
```

```
while true
```

```
do
```

```
    read -p "Enter a word (type 'stop' to quit): " word
```

```
    if [ "$word" = "stop" ]; then
```

```
        break
    fi
    echo "You entered: $word"
done
```

Case variable in

1)

```
;;
```

20

```
;;
```

***)**

```
;;
```

Esac

Loop:

```
#!/bin/bash
```

```
for var in list_of_items
do
    # commands using "$var"
done
```

Iterate over specific items:

```
for fruit in apple banana cherry
do
    echo "I like $fruit"
Done
```

C-style loop:

```
for (( i=0; i<5; i++ ))
do
    echo "i = $i"
done
```

Function:

Defining a function

There are two (common) syntaxes in Bash:

```
my_function () {
```

```
# commands  
}
```

Or

```
function my_function {  
    # commands  
}
```

Ex:

```
#!/bin/bash
```

```
hello () {  
    echo "Hello, world!"  
}
```

hello

Passing Arguments:

```
add_two () {  
    local a=$1  
    local b=$2  
    echo "Sum is $((a + b))"  
}
```

```
add_two 3 5
```

Return value:

```
get_name () {  
    echo "Alice"  
}
```

```
NAME=$(get_name)  
echo "Name is $NAME"
```